

Священная книга Oscarina

Игорь Казанова

Содержание

Глава 1 Что такое Oscarina?	1
Ещё одна!	1
Остальные	1
Мы	2
Культурная проблема	3
ISTQB, где ты?	3
Возврат к основам	4
Настоящая болевая точка	4
О «творчестве»	4
В чём здесь наука?	5
Видение	6
Суверенная грамматика	6
Рождённая практикой	6
Внедрение	6
Глава 2 Первые отзывы	8
Первые претензии	8
Показная сложность	9
Философия	9
KISS	9
Что это — быть «подрывным»	10
Неподкупная	11
Первый дельный отклик	12
Настоящая взаимовыручка	12
Почему не Playwright?	12
Размежевание	13
Анти No-Code	13
Анти-разработчики	14
Анти-«нации-стартапы»	14
Анти-хайп	15
Анти-slipologists	16
Глава 3 Первые шаги	18
Отказ от ответственности	18
1. Настройка проекта	18
2. Адаптеры	18
2.1 EnvGetters	18
2.2 Act	20
2.3 TestCampaign	20
2.4 TestSuite	21

2.5 MatchPage	22
3. Пишем первый POM	22
3.1 SeleniumTitleMixin	23
3.2 Возврат self	23
4. Пишем соединители	23
5. Пишем первый тестовый сценарий	24
6. Создаём тестовый набор	25
7. Создаём тестовую кампанию	25
8. Создаём тестовый цикл	25
9. Запуск проекта	26
Глава 4 Первые сценарии	28
Конец недопустимых состояний	28
Act и drive_page	28
match_page	30
Повторения	32
Фрагменты	33
Псевдонимы	34
Глава 5 Первые дзюцу	36
Наборы данных	36
Smoke-тесты	38
Setup и teardown	38
Жизненный цикл	38
Паттерн Proxy	39
Реактивное программирование: HET	40
Архитектурный ответ	40
Вызовы API и блокировки	41
Профиль браузера	41
Глава 6 Первые реальные препятствия	43
Случайные ошибки сервера	43
Случайные ошибки в пределах шага	44
Случайные ошибки Selenium	46
Дискретные случайные ошибки	46
Использование JS	47
Отпечатки	48
Отчёт	48
HumanizedDriver	48
Гейзенбаги конкурентности	48
Глава 7 Расширяемость	50
ValidationChain	50
Сцепление инвариантов	50

Сцепление валидаций	50
Переиспользуемые инварианты	50
Пользовательские утверждения	51
Безопасность типов	51
Success и Failure	51
Плагины	53
Расширяемая грамматика	53
Глава 8 Использование Ocarina с AI	54
Три духовных камня	54
CLAUDE.md	54
skills/	54
Ревью (14)	54
Анализ (4)	55
Чёрная шляпа (6)	55
Понимание (4)	55
Выбор (3)	55
Авторство (9)	56
Рефакторинг (2)	56
Состояние (1)	56
Настройка (2)	56
Запуск (1)	56
Повторяющиеся цепочки	57
Дисциплина	57
Чего эта схема не делает	57
Открытые ресурсы	57
Глава 9 From Ocarina to Igor	59
Чтобы пойти дальше	59
Ресурс, открытый для LLM	59

Что такое Ocarina?

Все фреймворки тестирования сделали одну и ту же ошибочную ставку. Ocarina — нет. Узнайте почему.

Ocarina создавалась с одной целью — сделать **автоматизированные браузерные тесты предельно простыми** и при этом оставить за пользователем **полный контроль**.

Ещё одна!

Остальные

Большинство фреймворков тестирования родились в мире, где барьер между «теми, кто пишет код», и «теми, кто описывает тесты», был настоящим — и встроенным в саму структуру.

Robot Framework попытался его **обойти** с помощью *DSL (предметно-ориентированного языка)* — со **своим форматом** и **своей экосистемой плагинов**. Тем самым RF по умолчанию навязывает собственные стандарты: это — прямая плата за его обещание.

Cucumber пошёл тем же путём с *Gherkin*: «естественным» языком, который на практике **сковывает всех и при этом никого по-настоящему не освобождает**. Цена: **постоянная прослойка-переводчик и рассинхрон между Gherkin и кодом**.

Все они поставили на одно и то же: **спрятать сложность**, чтобы навести мост между разными специалистами.

Итог: нетехнические специалисты так и остаются зрителями, а технические оказываются **в плену инструмента, который сами никогда бы не выбрали**.

Самая большая плата — это гонка на дно: меньше возможностей и «гибкость», которой можно добиться, только *воюя со своими инструментами, а не пользуясь решениями.*



Незачем тянуть за верёвку, которую тебе протянули, если в центре — всё тот же гордиев узел.

Мы



Лучше в одиночку, чем в дурной компании.

Oscarina ставит на обратное: этот барьер исчезнет. Это надуманный спор, вокруг которого каждый сам затянул петлю у себя на шее — из до неприличия сложных инструментов, выданных за «решения». Последствие: **операционная катастрофа**, стоит лишь возникнуть задаче, которую не выразить в рамках «фреймворка», на деле НЕ универсального.

И хуже всего: **все эти технологии так и будут катиться в ту же сторону**. НИ ОДНА не пойдёт на этот *разворот*, потому что он потребовал бы смены парадигмы — **возврата к основам, который напрямую противоречит всему их ценностному предложению**.

И всё же потребность никуда не делась — потребность в **читаемом, трассируемом и гибком** тестовом коде в его максимально **сыром** виде.

С AI и такими инструментами, как *Claude Code*, эта ставка крепнет день ото дня.

Мост между техническими и нетехническими специалистами — больше не слой абстракции.

Это сам AI. AI, работающий с сырыми данными.

Культурная проблема

Есть ещё одно слепое пятно, о котором почти не говорят: **методология**.

ISTQB, где ты?

ISTQB и профессиональные тестировщики десятилетиями выстраивали точный, проверенный на практике словарь: *циклы тестирования, кампании, наборы тестов, тестовые случаи, тестовые шаги*. Чёткая иерархия, призванная **организовывать, прослеживать и направлять** качество ПО.

Инструменты автоматизации это наследие **почти полностью проигнорировали**.

pytest, Jest, Mocha... все они — **гибридные помеси**, где тестировщику приходится учиться думать как разработчик и где на одном языке толком не говорит никто.



Этот «метод двух языков» — провал.

Возврат к основам

Oscarina на такой компромисс не идёт.

Её структура выстроена **напрямую и только** по методологии тестировщика. Каждое понятие в коде соответствует понятию из предметной области тестирования. Ничего не заимствовано, ничего не переименовано, никаких «и так сойдёт».

И раз уж Oscarina идёт в своей ставке до конца — **она полностью автономна**. Никаких плагинов для *pytest*. Никакой принудительной интеграции в чужую экосистему. Oscarina — это *batteries-included*: для работы ей больше ничего не нужно, и это осознанный выбор.

Настоящая болевая точка

О «творчестве»

Всех колотит от вопроса *как*: интерфейсы, абстракции, «красивые» DSL. А *почему* тем временем теряется из виду.

Oscarina выбирает обратное.

Весь её замысел и вся эта Священная Книга сосредоточены на **«почему»**: на реальных проблемах.

А не на абстракции, которой пользуются *«как заблагорассудится»*.

Ну а *как*?

Ответ прост: Oscarina **плотная, сразу готова к работе и строгая**. Сделана так, чтобы и люди, и

LLM без труда понимали её суть и то, как ею пользоваться.

В чём здесь наука?

Здесь всё держится на **статическом фундаменте**:

- типы,
- дженерики,
- функциональное программирование.

Oscarina самой своей конструкцией мешает использовать себя неправильно: последнее слово — за компилятором.



Просто алгебра.

📖 Слово «алгебра» происходит от арабского الجبر (al-jabr) — «воссоединение разрозненных частей», «вправление переломов».

Большинство инструментов отталкиваются от человеческой грамматики, чтобы якобы её «формализовать».

А потом выясняется, что машина не умеет читать её как есть.

И тогда сверху начинают громоздить «адаптеры».

А мы зовём это не формализацией, а принятием желаемого за действительное.

Oscarina, разумеется, поступает **наоборот**.

Именно это и делает её **стабильной и расширяемой одновременно**.

Видение

Суверенная грамматика

А что, если отдавать свою грамматику на откуп «стандартам» изначально было плохой идеей?

Настоящая проблема E2E-тестирования **вовсе не в том, чтобы навязать «лучший» новояз.**

Короткий ответ: Ocarina **расширяема**. Любые глаголы и союзы создаёшь сам — и всё это подчинено **строгим правилам, которые удерживают систему глубоко согласованной.**

Остаётся одно — обеспечить **трассируемость и надёжность.**

Рождённая практикой

В Ocarina наблюдаем каждый шаг.

Путь ошибки прописан явно. Отчёт о тестировании **рождается из кода сам собой.**

Никакого переусложнения. Никаких лишних зависимостей.

Просто нечто **компактное, читаемое и сделанное на века.**

А самое сильное — в том, что **Ocarina ничего не изобретает: Ocarina возвращается к основам.**

Внедрение

В самых крайних случаях Ocarina вообще не требует установки.

Скопируй, подгони под себя, запусти.

Аудировать нечего.

Для команд, скованных *политиками безопасности*: код компактный — его реально проверить за полдня. Ничего не спрятано. Единственные внешние зависимости — в плагинах постобработки; не подошёл какой-то из них — удаляешь, и больше ничего не ломается.

На практике консультант может прийти к заказчику с Oscarina в кармане — *почти* ни у кого не спрашивая разрешения.



Устают не люди — их изнашивают отжившие струны.

И именно поэтому она и существует — чтобы вернуть тестировщикам их **независимость**.
И всё это — **в одной жемчужине, вобравшей в себя самую суть**.

Первые отзывы

Первое, в чём упрекнули Oscarina. Угадайте, что именно.

С самого начала Oscarina без лишнего шума показывали профессионалам отрасли — самым разным, из всех слоёв.

Первые претензии

Первая «критика» часто звучала одинаково: *«Ты хвастаешься тем, что и так проще некуда».*

Некоторые на этом и остановились.

Другие шли дальше — пытались объяснить, что такое, по их мнению, «настоящая сложность».

Но никто из них не смог бы сделать то же самое.

Самое забавное — что **мне даже не пришлось самому заводить речь о сложности.** Я просто показывал пару тестовых сценариев на Oscarina — и реакция следовала мгновенно.

Это всего лишь проявление оперантного обусловливания у 99% людей в нашей отрасли.



Прикрутить к непонятной машине ещё одну шестерёнку — похоже, любимый фетиш многих инженеров моджо.

Таким хватило одного — открыть капот.

Начнём с главного: дело не в хвастовстве.

Oscarina — **не плод показухи**, в отличие от того, что обычно выдают многие «инженеры». Она — просто плод того, что *возникает само собой*.

Нарциссизм проявляется в том, чтобы постыдно усложнять всё подряд — с единственной целью: *господствовать*. А вовсе не в том, чтобы набираться мастерства и упрощать процессы, ведь итог тут один — полный отказ всего: хаос.

Так же быстро нарциссов вычисляют и в *казино*: чем ярче мигает автомат, чем «сложнее» он на вид, тем сильнее нарциссу хочется сесть за него и доказать, что он «умнее автомата».

Узнать их можно и по *неумению мириться с провалом*. Они вечно норовят показать, как именно ОНИ «продумали всё до мелочей».

С нарциссом — никогда никакого красного, только зелёное, только «готово в прод».

Последствия тут катастрофичны — да ещё и прямо враждебны самому складу мышления, на котором держится тестирование ПО: нарциссизм не приносит здесь ничего хорошего.

Да и вообще он нигде не приносит ничего хорошего — только *ад на земле*.

Oscarina — полная противоположность всему этому.

Показная сложность

Настоящая сложность не выставляет себя напоказ — её чувствуют. В стабильности, в расширяемости, в том, что не ломается.

Сделать что-то чудовищно неудобное в работе — в этом **нет никакой пользы**, кроме одной: доказать, что оно сложное. **Об этом никто не просит.**

В нашей отрасли серьёзная беда: сложность здесь принимают за знак солидности.

Эта сложность **существует с одной целью — «впечатлить коллег»** и не добивается ничего, кроме одного: **превзойти самого себя в глупости.**

Блестать не в ту сторону — хуже обычной посредственности, и вот единственное «достижение», какое тут можно разглядеть: поздравляем, вы успешно забрались на вершину **пирамиды бреда**, где уже не осталось места сомнению в себе!

Вот ещё почему **KISS** (*Keep it simple, stupid*) так превратно понимают в индустрии. Многие считают, будто «простое» значит «примитивное». Понять принцип ещё неправильнее было бы трудно.

Вот и получаем в итоге худшее из двух миров.

И как при таком раскладе можно называть себя *продуктивным*? Серьёзно?

Вы. Вообще. Не. Понимаете. Что. Такое. «Чистый. Код».

Вы даже не пробовали его писать!

Философия

KISS

KISS — в самом сердце Oscarina.

И если, прочитав свой первый тестовый сценарий, кто-то реагирует так: *«да это же проще простого...»* — обидно было бы услышать в этом критику: это ровно тот комплимент, которого мы и добивались.

Спасибо.

Чтобы прийти к этому, речь никогда не шла о том, чтобы «сиять» ярче других.

Да это с самого начала и не было настоящим вызовом.

Вопрос стоял просто: **что мне на самом деле нужно?**

У кое-кого нашлись свои идеи — и притом весьма «креативные»:

- Выкрутить реализацию ROP (*Railway Oriented Programming*) в Ocarina так, что от неё не осталось и следа смысла, — при том что они и сами не знали, что такое ROP;
- Напихать «*hooks*» и прочих так называемых «*ninja techniques*» прямо в середину тестовых шагов;
- Отказаться от типизации;
- Переписать всё на Rust ради «производительности»;
- Навязывать мне своё безграмотное понимание ленивых вычислений и *IoC* как истину в последней инстанции;
- «Объяснять» мне разницу между императивным и декларативным программированием, неся при этом полную чушь;
- «Заводить» со мной разговор про *event-driven programming* — всё с тем же бредом;
- И, что хуже всего, — разглагольствовать про чудовищную теорию «декларативного объектно-ориентированного программирования»;
- И так далее.

Даже один человек, которого я до того очень уважал, начал меня изрядно доставать. Он вообразил, будто может меня чему-то научить: держался высокомерно, неприятным тоном втолковывал мне, «чего не хватает», — хотя всё, о чём он толковал, не просто стояло в дорожной карте (пусть и намеренно там скрытое по стратегическим соображениям), но и заходило куда дальше всего, что он вообще способен вообразить.

Я — я молчал, ничего не ответил, просто отправил *gero*.

Он же — сразу «знал всё» лучше всех остальных.

Что это — быть «подрывным»

Ну, я сейчас прохожу *YC*, и мой клиент — *IBM*.

Ага, а я знаком с королевой Англии.

Я вам не рассказывал про якудза?

Я закончил топовый технический вуз — и с тех пор не узнал ничего нового.

Ну что ж: процитирую-ка я девиз первой индонезийской хакерской команды, на которую я наткнулся ещё ребёнком: «We Can Do All What You Can't Do».

Я — тот баг, который тебе не убить.

И я здесь не ради *престижа*. И даже не ради *прибыли*.

Я здесь ради нашего **сообщества**.

In the Lulzboat, salute, bitch, and show some respect.^[1]

Ты — ты был бы тем самым *lamer'*ом, тем пацаном, которому просто хотелось покрасоваться и который запускал PoC с Exploit-DB у себя в спальне, как *rookie*. Чтобы что-то доказать, выставить себя напоказ, показать нам, до чего ты *in*. Я — я тот пацан, который впитал всё и

вырос на этом.

Чёртовы скиды.

Чёртовы нормы!

Вот кто мы: от Zone-H — к жизни приличной.

Из подземных сортиров интернета — к жизни, где получается тихо приносить пользу.

Ты бы такого нипочём не пережил.

Из Ада — туда, где мы сейчас.

Нас спас научный поиск, спасли прекрасные ценности, спас неблагодарный труд. Не тот труд, что заставляет блистать. А тот, что передают по наследству люди, умеющие разглядеть потенциал и спасти его, пока не стало слишком поздно.

НИКТО нас не спас — мы спаслись САМИ.

Благодаря тому, что говорит НАУКА.

Мы могли стать кровожадными — а стали жадными до мастерства. Мы — люди, отдавшие жизнь этим экранам, этой науке, пока ты травил подростков в чатах по Dota или занимался бог знает чем — вечно красуясь, вечно доказывая, что ты самый красивый, самый сильный, самый умный, самый властный.

Мы — мы чокнутые, но мы останемся в сети до самого конца!^[2]

Мы — настоящая СЕМЬЯ!

И мы НЕ СПИМ — НИКОГДА!

«Это просто полный аутизм.»

«Из тебя никогда ничего не выйдет.»

«Ты псих.»

«То, что ты делаешь, — полная глупость.»

«Да то, что ты пишешь, вообще лишено смысла.»

THE FUCK YOU THINK THIS IS?

You HOLLOW!

YOU HOLLOW, YOU UNDERSTAND ME?

YOU'RE WORTHLESS, YOU'RE FUCKING TRASH!^[3]

(btw: RIP, DG descendant...)

**ТЕБЯ ЗАМЕНЯТ, И ТЫ ВЕРНЁШЬСЯ ТРАВИТЬ РОВЕСНИКОВ В ДОТЕ И ДРОЧИТЬ НА МАКРОСЫ VBA — КАК КУСОК ДЕРЬМА,
КОТОРЫМ ТЫ И ЯВЛЯЕШЬСЯ!**

ГРЁБАНЫЙ ТЫ НЕКОМПЕТЕНТНЫЙ УБЛЮДОК!

ГРЁБАНЫЙ ТЫ ПАРАЗИТ!

ГРЁБАНАЯ ТЫ БЕЗДАРЬ!

YOU FUCKED WITH US!

Неподкупная

И к моему ответу прилагается цитата Дэвида Хейнемайера Хансона (DHH): «Fuck You.»^[4]

Да. Exactly: Fuck You. Вот и всё.

МНЕ НЕ ИНТЕРЕСНО программировать «вот так», а владелец Oscarina — Я.

Передать Oscarina — значит **передать машину, которую я полностью, своими руками обслуживал сам и для себя, а потому очень бережно.** И всё же она передаётся как есть — и такой и останется.

Это *моя* машина.

Я направил в неё всю свою ярость — чтобы вложить в неё всю свою любовь.

У Oscarina есть направление.

Кто хочет увести её в другую сторону со своим *wishful thinking* — волен сделать форк и никогда со мной не связываться.

Вносить вклад в проект с открытым исходным кодом потому, что это «круто», — совершенно незрелый подход, и мириться с этим явлением — значит наносить реальный вред.

Ни один настоящий контрибьютор не делает это «ради фана» — только ради *согласования личных интересов*.

Oscarina — не *напыщенное* решение и никогда им не станет.

Oscarina была и **ОСТАНЕТСЯ** решением для реальных, конкретных задач.

Не согласен — **возвращайся на [r/unixporn](https://r.unixporn.com/)^[5], где тебе и место.** ✈

Первый дельный отклик

Профессионал, который **по-настоящему** изучил структуру проекта, отреагировал так: «*Похоже на один из моих старых Selenium-проектов, а я их терпеть не мог.*» Это **ровно** тот отклик, ради которого Oscarina и создавалась.

Первым делом он приравнял POM к Selenium. Резонно.

Вот только POM работает с любой технологией автоматизации.

«Старомодно»? Ещё как. И что? Что дальше?

Разбирая собственные претензии, он увидел, как Oscarina их снимает: «Стоп, и это всё?»

Но на этот раз — уважительно кивнув. Он признал: пора перестать рваться в *самые умные в комнате*. С какого-то момента именно это и убивает проект.

Настоящая взаимовыручка

Вместо того чтобы уходить в *nerdy*, Oscarina занята другим — решает *мелкие задачи*, не порождая крупных.

Отсюда и вывод: «Oscarina практична».

В этом и вся цель Oscarina — в её практичности.

Вот так работаем **мы** — и этого правда достаточно, чтобы проникнуться философией Oscarina.

Не нужно быть хакером, не нужно было столько всего выстрадать.

Это инструмент, который мы дарим, — для увлечённых, по-настоящему увлечённых людей.

Для тех, кто хочет строить, а не разрушать.

Для цельных, культурных людей, которые знают, кто они.

Для таких, как мы, в сущности.

Хоть раз — давайте держаться вместе.

Подальше от тех, кто разграбил то, чем мы были.

Почему не Playwright?

Следующий его вопрос: «Почему не Playwright?»

Тоже по делу.

Oscarina агностична, так что подключить Playwright несложно.

Единственное ограничение: Oscarina никогда не будет поддерживать `async/await`.

Более того: начиная с **Oscarina 1.1.3**, вручную подключать уже ничего не нужно. Полноценный адаптер Playwright поставляется вместе с фреймворком, а [готовый пример](#)^[6] это подтверждает.

Размежевание

В наше время проекты запускают так же запросто, как сбегали бы за пачкой сигарет. Пробуют — потому что это «модно». И вот все копируют друг друга, оформляют подписки на продукты друг друга, лишь бы взаимно накручивать себе цифры и сбивать друг другу *Stripe dispute rate*.

Они считают себя умными, но на деле **это прекрасно понимает каждый VC и каждый LP**, и каждый играет свою роль в *fool's game*.

Но есть то, чего им не понять никогда. В них нет ничего *«underground»*: они всего лишь **дети в кризисе идентичности**. Дети, которым по-доброму обменяли цветные карандаши на *Powerpoint*.

Урок этим дилетантам: любой по-настоящему *«cutting-edge»* проект, каким бы он ни был, начинается с **памфлета**, уходит корнями в **идентичность** и проходит через стадию настоящего ***anti-marketing***. Но поскольку вы все *боитесь опозориться перед мамочкой*, вы эту стадию так и не проходите.

Что до меня — мне нечего терять в смысле репутации, никакой репутации под этим именем я и не строю. Есть только послание, которое нужно донести — как есть, без фильтров.

Пора надавать пинков всем вашим *Juicero Presses*.

С той самой **понимающей усмешкой** — усмешкой тех, **у кого наконец хватило духу сказать СТОП**.

IT — это **то, что вернуло в нашу жизнь улыбку**, а не то, что травило нас псевдо-«паттернами», один другого идиотичнее и недоступнее.

Вы думали, информатика умрёт?

Куда там.

Анти No-Code

Код — это сырые данные. Их можно проверить. Можно изучить. **Белый ящик**.

Ровно то, с чем AI умеет работать с самого своего зарождения.

В своих замкнутых кружках мы это уже распробовали — ещё до *IntelliCode* (2018). Уже в 2013-м у нас: приватный плагин для *Emacs* (вот чёрт), собранный одним из наших безумных учёных — R. Никто не мог осознать его уровень в *reverse engineering* — и его лаконичность. Да и в других областях — то же самое... по правде, не было ничего, в чём он был бы плох, и он обладал поразительным даром всегда писать *ровно то, что нужно было написать*.

Конец мифа: у него был собственный *AI-autocomplete* и *auto-review*. 2013 год.

Он отвергал бóльшую часть сгенерированного кода, но говорил так: *«Я не против удалить 15 000 строк, если это избавляет меня от написания тысячи — вручную.»*

К 2013-му дело было уже не в том, чтобы быть бездумной «кодящей» обезьяной. Дело было в том, чтобы развивать себя — понять, как машина, вдохновлённая нашим же мышлением, поможет нам подняться над всем этим тошнотворным, обречённым на смерть ООР.

Но «мудрёный» ООР падёт не первым и не последним.

Вычислительная техника развивается в основном в одну сторону: кто по доброй воле выберет сегодня Windows 95?

И всё же λ -исчисление (1930) настолько мощно, что сегодня снова в центре внимания — как и AI (первая формализация искусственного нейрона — 1943, McCulloch & Pitts). Небывалый случай в истории IT.

Последние достижения AI перетасовали колоду в мире SaaS, который существует с одной целью — спрятать сложность. По-настоящему полезные SaaS-продукты — а не бесконечные агрегаторы этой помпезной «цифровой трансформации» — всегда на правильной стороне Истории. Какими бы уродливыми они ни были.

Компании из экосистемы *Prisma* или *Vercel* тратят ~\$200 в токенах на каждого *vendor*'а, от которого избавляются. Выходные за *vibe coding* — и они машут на прощание раздутым, переоценённым, неповоротливым SaaS-платформам в пользу *internal development* и возврата к **сырым данным**.

Декабрь 2025-го: Ли Робинсон, экс-Vercel, знакомое по недавним презентациям *Next.js* и *Turborepo* лицо, сообщает, что отказывается от *Sanity* — CMS, которую до того выбирал *Cursor*. В пользу возврата к **сырым данным**, к простым файлам *Markdown*. Несколько токенов, немного грамотного *vibe coding*'а. Спасибо, до свидания.

Code is Law. Код *подменяет* собой Закон. Лессиг, 1999-й, — как крик предостережения.

Затем, в 2015-м, его подхватывает Ethereum — наоборот, как политический идеал. Значимый шаг вперёд в мире валют, отчеканенных из факторизации чисел.

Анти-разработчики

2016-й: взлом The DAO, децентрализованного инвестиционного фонда на \$ETH.

Почему?

Баги. Проклятые баги — из-за проклятых разработчиков.

Идеологический откат почти на 20 лет. Контрвласть выбрала лёгкий путь — «не понимать»; тот самый выбор, что позволил своре некомпетентных проскользнуть незамеченными.

А ведь всё просто: в отличие от *Исследователей*, инженеры и их приятели из бизнес-школ полагаются на *wishful thinking*.

Самые «престижные» школы научили их одному — выдавать пустозвонство за «инженерию»; те же, кто действительно знает своё дело, зовут это *программистской астрологией*.

Анти-«нации-стартапы»

И снова этот поседевший спор о «демократическом управлении кодом».

Но кто в это лезет?

Кадровики?

СЕО?

Пресейл-менеджеры?

2022-й: ChatGPT. И всё же три года спустя основатели стартапов по-прежнему продают *No-Code*, «усиленный AI». Чистый *cash burn* в погоне за *рыночной аномалией* — не более того. *Альфа* инвесторов... иначе говоря — их *казино*.

Не будем никого называть — тем более что их результаты ждут во II квартале 2026-го. Скажем лишь, что с точки зрения *фундаментального анализа* в этой ставке нет смысла. При этом и зла им желать незачем. Не первая на нашей памяти компания с треском провалится — и не последняя на удивление всем преуспеет.

Факт остаётся фактом: код — самый суверенный актив, какой у нас есть, и ценностное предложение, вся суть которого — отнять его у нас, заставляет нас брезгливо морщиться. Настоящая проблема — разработчики, которым мы позволяем долбить по клавишам, не понимая, что творят, как дрессированным обезьянам. Сегодня *Claude Code* работает лучше 99% из них — и они воют. *Собака лает — караван идёт*.

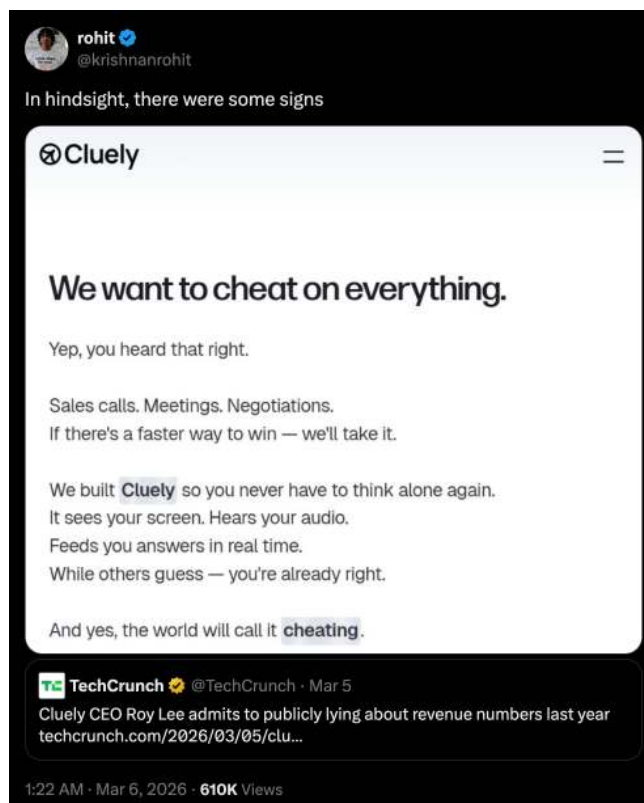
Истинная добавленная ценность человека ещё никогда не была обозначена так чётко: *вкус и чувство ответственности* — качества, которых этим «профессионалам» вызывающе недостаёт, ровно как и тем, кто *продаёт* то, в чём не разбирается.

Анти-хайп

В том же декабре 2025-го стартап, чьё имя останется неназванным, подавали в некоторых СМИ как «*AI No-Code-мечт, который унижает Selenium и заставляет BrowserStack дрожать*». Ни больше ни меньше. Удачи им: предпринимательство — это мир ставок и случая, где лучшие идеи попадают на кладбище ничуть не реже самых вздорных. Фундаментальный анализ там проигрывает регулярно.

Однако ничто из этого не собьёт *Oscarina* с видения, которое складывается уже больше 15 лет, вдали от высокомерия всех этих *детей хайпа*.

Что бы ни случилось, мы всё же желаем им судьбы, что избавит их от унижения — столь же стремительного и зрелищного, как то, что постигло основателя *Cluely*, — и предлагаем им умерить своё высокомерие.



CEO *Cluely* Рой Ли признаёт, что в прошлом году публично врал про цифры выручки. А затем Кришнан Рохит указывает: их ценностное предложение с самого начала было мошенничеством.

Сегодня гонка за звание самого крупного мошенника приносит всё меньше и меньше прибыли.

И мы этому рады. И снова, вместе с AI — этим так недостававшим нам цифровым телом, — прокричим это так же громко, как кричали «**HACK THE PLANET**» ещё в 99-м: **CODE IS LAW**.

Анти-slipologists

Слишком долго **программное обеспечение оставалось заложником** меньшинства, «одного процента», который счёл уместным превратить его в **площадку для посвящённых**: «гикивские трюки», «ниндзя-техники», «объектная ориентированность». **Вердикт обжалованию не подлежит: оно не выдерживает — или выдерживает едва-едва.**

То, что занимает наши умы с 1930-х, **со времён изобретения λ -исчисления**, наконец масштабируется до тех пределов, каких нам всегда хотелось.

Видела ли наша индустрия когда-нибудь столь изящную формализацию — так глубоко укоренённую в собственном наследии?

Недавнее развитие *системы типов* Python — одна из главных причин, по которым Ocarina вообще возможна.

И это при том, что никакое это не «будущее»: **это устоявшаяся наука — просто в нашу экосистему она пришла до жестокого поздно.**

Ровно как и AI, который тихо обращает способность «одного процента» вредить в анахронизм.

Благодарность, которой заслуживают эти технологические достижения, так же безмерна, как ярость тех, кто против них восстаёт, — тех, кого в наших краях мы ласково зовём *slipologists*.

См. также: *Haters, Пола Грэма*.^[7]

По всем этим причинам я принял решение **НИКОГДА не тратить время на споры, которые того не стоят**. Спорить мне нравится с людьми, **нацеленными на решение задач и наделёнными здравым смыслом**.

Что до остальных: **бессмысленные PR — в красное, бессмысленные issue — в серое, без сожалений**.

Просто некоторым людям не дано мне понравиться — что ж, так тому и быть.

Oscarina — *первый из моих проектов, который я открываю публике, но на подходе и другие — в том числе куда более интересные*.

Из андеграунда,
Отныне и навсегда.

Peace out.^[8]

"Идиот восхищается сложностью, гений восхищается простотой, а физик старается всё упростить. Для идиота: чем что-то сложнее, тем сильнее он будет этим восхищаться; если ты сделаешь нечто настолько запутанное, что ему не понять, он сочтёт тебя богом — ведь ты сделал это таким сложным, что не понять никому. Вот так и пишут статьи в академических журналах: всё стараются усложнить до того, чтобы люди приняли тебя за гения."

— Терри Дэвис, "самый умный программист из всех, что когда-либо жили"

"Чтобы обрести знание, каждый день что-то прибавляй. Чтобы обрести мудрость, каждый день что-то отнимай."

— Лао-цзы

[1] <https://www.youtube.com/watch?v=PCW6BkSp1Sc>

[2] <https://soundcloud.com/spokepp4l/retarded>

[3] <https://soundcloud.com/queed-inc/true-colors>

[4] <https://world.hey.com/dhh/i-won-t-let-you-pay-me-for-my-open-source-d7cf4568>

[5] <https://www.reddit.com/r/unixporn/>

[6] <https://github.com/mojo-molotov/ocarina-with-playwright>

[7] <https://paulgraham.com/fh.html>

[8] <https://soundcloud.com/ytcracker/ytcracker-robots-will-definitely-take-your-job>

Первые шаги

Дело не в пункте назначения, а в самом пути.

Отказ от ответственности

Примечание: эта книга призвана помочь вам освоиться с проектом **ocarina-example** — он остаётся **источником истины**, к которому следует обращаться в любой ситуации.

 *The Ocarina Holy Book* — это НЕ «plug-and-play» и никогда им не будет. Чтобы пользоваться Ocarina, нужна достаточная зрелость. Поэтому мы остановимся только на том, что и правда может оказаться непростым.

Эта страница объясняет *путь*. А дальше понадобится практика.

 [Возьмите за образец канонический пример.](#)^[1]

 [Также доступен с адаптером Playwright.](#)^[2]

1. Настройка проекта

Создайте новый проект Python, затем установите необходимые зависимости:

```
pip install selenium
pip install ocarina
```

Затем создайте структуру папок.

2. Адаптеры

Ocarina построена вокруг системы адаптеров, которые пишет сам пользователь. Они позволяют настроить фреймворк под ограничения и соглашения конкретного проекта.

Основные адаптеры, которые нужно создать:

- **act** (*требуется*)
- **test_campaign** (*требуется*)
- **test_suite** (*требуется*)
- **env_getters** (*опционально*)
- **match_page** (*опционально*)

2.1 EnvGetters

EnvGetters в Ocarina централизует и типизирует доступ к переменным окружения. Он делится на две категории:

- **Creds:** пары «логин/пароль», представленные неизменяемыми словарями.
- **Values:** отдельные значения (строки).

```

type _CredsKeys = Literal["dashboard"]
type _ValuesKeys = Literal["igor_xxx_key", "xxxxx_url"]

def _load_env() -> None:
    from dotenv import load_dotenv

    load_dotenv()

_DEFAULT_EFFECTS = (_load_env,)

class _EnvGetters(EnvGetters[_CredsKeys, _ValuesKeys]):
    def __init__(self, *, effects: Effects) -> None:
        for effect in effects:
            effect()

        super().__init__(
            credentials={
                "dashboard": MappingProxyType(
                    {
                        "login": os.environ["DASH_USERNAME"],
                        "password": os.environ["DASH_PASSWORD"],
                    }
                ),
            },
            values={
                "igor_xxx_key": os.environ["IGOR_XXX_KEY"],
                "xxxxx_url": os.environ["XXXXX_URL"],
            },
        )

def create_env_getters(*, effects: Effects | None = None) -> _EnvGetters:
    """Create a fresh EnvGetter instance."""
    if effects is None:
        effects = _DEFAULT_EFFECTS
    return _EnvGetters(effects=effects)

```

Когда адаптер готов, получение значения или учётных данных выглядит так:

```

xxxxx_url = create_env_getters().get_value("xxxxx_url")
dashboard_creds = create_env_getters().get_credentials("dashboard")
print(xxxxx_url)
print(dashboard_creds["login"])
print(dashboard_creds["password"])

```

Примечание: допустимые ключи задаются двумя типами: `EnvGetters[_CredsKeys, _ValuesKeys]`. Если нужен только `.get_value()`, достаточно типизировать `_CredsKeys` как `Never`. И наоборот: `_ValuesKeys` следует типизировать как `Never`, если нужен только `.get_credentials()`.

После этого наши аксессоры строго типизированы. Например:

```
xxxxx_url = create_env_getters().get_value("x")

# error: Argument 1 to "get_value" of "EnvGetters" has incompatible type
# "Literal['x']"; expected "Literal['igor_xxx_key', 'xxxxx_url']"
```

2.2 Act

В Oscarina **act** — это глагол, которым выражают каждый отдельный шаг тестового сценария. Его реализацию намеренно оставили на усмотрение пользователя — по причинам, о которых речь пойдёт дальше в этой книге (*hooks*).

Его минимальная форма выглядит так:

```
def act(pom: TPOM, action: Callable[[TPOM], TPOM]) -> ActionStart[TPOM]:
    """Act on a page."""

    return create_act(
        pom,
        action,
    )
```

2.3 TestCampaign

Адаптер **TestCampaign** намеренно минималистичен. Единственное, что Oscarina не может вывести сама, — это **количество рабочих потоков**, то есть сколько браузеров запускать параллельно в рамках кампании. А поскольку этот параметр можно передать и напрямую через CLI, достаточно совсем небольшого адаптера:

```
@final
class TestCampaign(OriginalTestCampaign[WebDriver]):
    """TestCampaign adapter."""

    def __init__(
        self,
        *,
        name: str,
        suites: Sequence[TestSuite[WebDriver]],
        max_workers: int | None = None,
        saturate_workers: bool | None = None,
    ) -> None:
        """Initialize the campaign."""
        if max_workers is None:
            max_workers = get_max_workers()

        super().__init__(
            name=name,
            suites=suites,
            max_workers=max_workers,
            saturate_workers=saturate_workers,
        )
```

Тип **WebDriver** (*Selenium* или другой) подставляется здесь:

OriginalTestCampaign[WebDriver].

И здесь: **suites: Sequence[TestSuite[WebDriver]]**

✓ Разумеется, сюда вставляйте ВАШ собственный **TestSuite**, а не встроенный в *Oscarina*.

2.4 TestSuite

Это адаптер, который важнее всего понять. **TestSuite** из коробки предоставляет множество параметров. Задача нашего адаптера — построить вокруг него **фасад**: одни значения раз и навсегда зашиты в код, другие — опционально, с разумными значениями по умолчанию.
Сужение.

Аналогично:

```
@final
class TestSuite(OriginalTestSuite[WebDriver]):
    """TestSuite adapter."""

    def __init__(
        self,
        *,
        name: str,
        tests: Sequence[Test[WebDriver]],
        drivers_pool: SeleniumWebDriversPool,
        create_logger: Thunk[ILogger] | None = None,
        copy_indicator: str = "+",
        put_space_after_copy_indicator: bool = False,
        autoscreen_on_fail: bool = True,
        saturate_workers: bool | None = None,
    ) -> None:
        """Initialize the TestSuite."""
        if create_logger is None:

            def _create_logger():
                return create_matching_logger(get_logger_mode())

            create_logger = _create_logger

        super().__init__(
            name=name,
            tests=tests,
            only_ids=get_only(),
            exclude_ids=get_exclude(),
            max_retries_per_test=8,
            create_logger=create_logger,
            drivers_pool=drivers_pool,
            copy_indicator=copy_indicator,
            put_space_after_copy_indicator=put_space_after_copy_indicator,
            autoscreen_on_fail=autoscreen_on_fail,
            take_screenshot=_take_screenshot_on_fail,
            transient_errors=transient_errors,
            saturate_workers=saturate_workers,
        )
```

Тип **WebDriver** (*Selenium* или другой) подставляется здесь:

OriginalTestSuite[WebDriver].

А также здесь: **tests: Sequence[Test[WebDriver]]**

И здесь: **drivers_pool: SeleniumWebDriversPool**

Временные ошибки

Концепция `transient_errors` играет в `TestSuite` центральную роль.

Такие ошибки считаются **шумом**: если тест упал из-за исключения, перечисленного в `transient_errors`, он автоматически перезапускается.

Максимальное число попыток задаёт `max_retries_per_test`.

Этот механизм делает прогон тестов устойчивым к *flakiness*. Тесты, которые перезапускаются часто, хорошо заметны в логах — и это позволяет тем, кто сопровождает проект, находить и устранять источники нестабильности: будь то неправильное использование Selenium, неподконтрольные условия среды или другие внешние факторы.

Only IDs и exclude IDs

Эти два параметра позволяют выполнять тесты выборочно.

Это фильтры по ID.

⚠ **Обязательно включите их в этот адаптер — иначе эти флаги CLI не будут обрабатываться.**

2.5 MatchPage

`match_page` — это оператор Oscarina, созданный для работы со страницами с недетерминированным рендерингом: cookie-баннеры, антибот-проверки, A/B-тесты и т. д.

Его логика проста: **любое выброшенное исключение трактуется как несовпадение и потому поглощается `match_page`**. Впрочем, часть исключений можно вывести из-под этого механизма — так, чтобы они нормально распространялись вверх по потоку выполнения.

Ради согласованности `transient_errors`, как правило, должны попадать именно в эту категорию: им положено распространяться, а не тихо проглатываться.

Адаптер создаётся так:

```
match_page = create_match_page(raised_exceptions=transient_errors)
```

3. Пишем первый POM

Паттерн POM (*Page Object Model*) — давно устоявшийся стандарт, и заново определять его мы здесь не станем.

Вот как создать первый POM на Oscarina:

```
@final
class Homepage(SeleniumTitleMixin, POMBase):
    """My homepage."""

    def __init__(self, *, driver: WebDriver, url: str = HOMEPAGE_URL) -> None:
        """Initialize homepage POM."""
        self._driver = driver
        self._URL = url

    def open(self) -> Homepage:
        """Open the page."""
        self._driver.get(self._URL)
        return self
```

```
def verify(self, *, timeout: float | None = None) -> Homepage:
    """Verify function."""
    try:
        if timeout is None:
            timeout = get_timeout()

        WebDriverWait(self._driver, timeout).until(ec.title_is("Welcome to my homepage"))

        WebDriverWait(self._driver, timeout).until(
            ec.text_to_be_present_in_element(
                (By.TAG_NAME, "h1"),
                "My homepage",
            )
        )
    except TimeoutException as exc:
        raise PageVerificationError from exc

    return self
```

Несколько моментов стоит разобрать подробнее.

3.1 SeleniumTitleMixin

Любой объект, унаследованный от `POMBase`, обязан реализовать метод `get_current_title`. `SeleniumTitleMixin` даёт эту реализацию прозрачно — вручную писать её не нужно.

Но этим его роль не исчерпывается: он ещё и задаёт атрибуту `_driver` тип `WebDriver` (Selenium), делая его **несовместимым с любым другим типом**. Попытка присвоить неподходящее значение сразу же вызовет ошибку типизации:

```
self._driver = "lol"

# error: Incompatible types in assignment
# (expression has type "str", variable has type "WebDriver")
```

Тем самым `SeleniumTitleMixin` ещё и работает как **страж типа**. Аналогичные миксины можно создать и для других технологий браузерной автоматизации.

3.2 Возврат `self`

Каждый метод-действие возвращает `self`. Это осознанное архитектурное решение Oscarina, которого следует придерживаться неукоснительно: оно позволяет сцеплять вызовы методов в цепочки и плавно компоновать сценарии.

4. Пишем соединители

Соединители — это тонкая, но важная прослойка ради читаемости сценариев. Они оборачивают вызовы методов POM в функции с понятными, явными именами:

```
def open_homepage(p: Homepage) -> Homepage:
    """Open my homepage."""
    return p.open()
```

```
def verify_homepage(p: Homepage) -> Homepage:
    """Verify we are on my homepage."""
    return p.verify()
```

Их также можно компоновать напрямую:

```
def open_then_verify_homepage(p: Homepage) -> Homepage:
    """Open my homepage, then verify it."""
    return p.open().verify()
```

5. Пишем первый тестовый сценарий

Все строительные блоки на месте.

Вот как собрать из них сценарий:

```
def open_and_verify_homepage(driver: WebDriver, logger: ILogger):
    """Open and verify my homepage."""
    on_homepage = Homepage(driver=driver)

    just_log_error = create_just_log_error(logger=logger)
    just_log_success = create_just_log_success(logger=logger)
    log_error_with_current_url = create_log_error_with_current_url(
        logger=logger, driver=driver
    )
    log_success_with_current_url_and_take_screenshot = (
        create_log_success_with_current_url_and_take_screenshot(
            logger=logger, driver=driver
        )
    )

    return [
        drive_page(
            act(on_homepage, open_homepage)
            .failure(just_log_error("Failed to open the homepage..."))
            .success(just_log_success("Opened the homepage!")),
            act(on_homepage, verify_homepage)
            .failure(
                log_error_with_current_url(
                    "Failed to verify the homepage...",
                )
            ),
            .success(
                log_success_with_current_url_and_take_screenshot(
                    "Verified the homepage!"
                )
            ),
        ),
    ],
]

test_homepage = create_selenium_test(
    name="Validate homepage",
    test_scenario=lambda driver, logger: Scenario(
        test_chain=open_and_verify_homepage(driver, logger)
    ),
)
```

```
)
```

Каждый шаг теста выражается через `act`, к которому прицепляют обработчики `.failure()` и `.success()`.

Затем сценарий оборачивается в объект `Test` с помощью `create_selenium_test`.

6. Создаём тестовый набор

Набор объединяет тесты, которые нужно прогнать на одном и том же пуле драйверов:

```
def create_my_first_suite(  
    *,  
    drivers_pool: SeleniumWebDriversPool,  
) -> TestSuite:  
    """Create my first suite."""  
    return TestSuite(  
        name="My very first suite with Ocarina",  
        tests=[  
            test_homepage,  
        ],  
        drivers_pool=drivers_pool,  
    )
```

7. Создаём тестовую кампанию

Кампания объединяет несколько наборов:

```
def create_my_first_campaign(  
    *, drivers_pool: SeleniumWebDriversPool  
) -> TestCampaign:  
    """Create my first campaign."""  
    return TestCampaign(  
        name="My very first campaign with Ocarina",  
        suites=[  
            create_my_first_suite(drivers_pool=drivers_pool),  
        ],  
    )
```

8. Создаём тестовый цикл

Цикл объединяет несколько кампаний. Это единица выполнения самого высокого уровня:

```
E2E_CYCLE_NAME = "My very first cycle with Ocarina"  
  
def create_my_first_cycle(drivers_pool: SeleniumWebDriversPool):  
    """Create my first cycle."""  
    return TestCycle(  
        name=E2E_CYCLE_NAME,  
        campaigns=[  
            create_my_first_campaign(drivers_pool=drivers_pool),  
        ],  
    )
```

9. Запуск проекта

Вот полная точка входа в проект:

```
if __name__ == "__main__":
    CliStoreSingleton().push(create_selenium_auto_cli_store())

    drivers_pool = create_selenium_drivers_pool(
        browser=get_browser(),
        driver_path=get_driver_path(),
        headless=get_headless(),
        wait_timeout=get_timeout(),
        max_size=get_max_workers(),
        profile_path=get_profile_path(),
    )

    def _post_exec(results: TestCycleResults) -> None:
        print()
        pretty_print_results(results, with_colors=True)
        if has_test_cycle_failed(results):
            sys.exit(1)

    with timing(prefix="Tests duration:"):
        bootstrap(
            post_exec=_post_exec,
            test_cycle=create_my_first_cycle(drivers_pool),
            run_plugins=lambda results: run_plugins(
                lambda: generate_docx_proof(
                    logs_root=get_default_log_dir() / E2E_CYCLE_NAME,
                    logger=create_matching_logger("terminal").set_domain_taxonomy(
                        "Generate DOCX proofs plugin",
                    ),
                    output_root=Path.cwd() / ".reports" / "tests_docx_output",
                ),
                lambda: generate_json_results(
                    results=results,
                    output_dir=Path.cwd() / ".reports" / "tests_json_output",
                    logger=create_matching_logger("terminal").set_domain_taxonomy(
                        "Generate JSON report file plugin",
                    ),
                ),
            ),
            exceptions_logger=PrintLogger()
            .set_prefix(
                lambda: concat_metadata(
                    format_utc_date_metadata_str, format_current_thread_metadata_str
                )
            )
            .set_domain_taxonomy(("Post-execution plugins",)),
        ),
    )
```

Процесс выглядит так:

1. Аргументы, полученные из CLI, помещаются в глобальное хранилище.
2. Создаётся пул драйверов: он управляет жизненным циклом веб-браузеров, работающих параллельно.

3. Определяется обратный вызов `_post_exec`: он срабатывает после тестов и плагинов, выводит результаты и завершается с кодом ошибки, если цикл провалился.
4. Вся загрузка происходит внутри таймера, измеряющего общую продолжительность выполнения. Таким образом, порядок выполнения такой: **цикл** → **плагины** → **post_exec**.

 Плагины — это отложенные функции, передаваемые в `run_plugins`.
`run_plugins` принимает `results` как аргумент —
и уже по одной сигнатуре функции сразу ясно, что выполняются они на этапе
постобработки, как только результаты готовы.

[1] <https://github.com/mojo-molotov/ocarina-example>

[2] <https://github.com/mojo-molotov/ocarina-with-playwright-example>

Первые сценарии

Язык не реакционен и не прогрессивен — он попросту фашистский; ибо фашизм не запрещает говорить, он принуждает говорить.

Конец недопустимых состояний

... Сделайте недопустимые состояния непредставимыми.

Разберём подробно, как `act`, `drive_page` и `match_page` работают при написании тестовых сценариев на Oscarina.

Act и drive_page

Канонический пример

Начнём с примера, который будем постепенно ломать:

```
def go_from_homepage_to_book_call_page_with_the_cta(driver: WebDriver, logger: ILogger):
    """Open and verify my homepage."""
    on_homepage = Homepage(driver=driver)
    on_book_a_call_page = BookCallPage(driver=driver)

    just_log_error = create_just_log_error(logger=logger)
    just_log_success = create_just_log_success(logger=logger)
    log_success_with_current_url_and_take_screenshot = (
        create_log_success_with_current_url_and_take_screenshot(
            logger=logger, driver=driver
        )
    )

    return [
        drive_page(
            act(on_homepage, open_then_verify_homepage)
            .failure(just_log_error("Failed to reach the homepage..."))
            .success(
                log_success_with_current_url_and_take_screenshot(
                    "On the homepage!"
                )
            ),
            act(on_homepage, click_book_call_page_cta)
            .failure(just_log_error("Failed to click on the 'Book a call' CTA..."))
            .success(just_log_success("Clicked on the 'Book a call' CTA!")),
        ),
        drive_page(
            act(on_book_a_call_page, verify_book_call_page)
            .failure(just_log_error("Failed to verify the 'Book a call' page..."))
            .success(
                log_success_with_current_url_and_take_screenshot(
                    "On the 'Book a call' page!"
                )
            ),
        ),
    ],
```



```

]

test_homepage_book_a_call_cta = create_selenium_test(
    name="Go from homepage to book a call page, clicking the CTA",
    test_scenario=lambda driver, logger: Scenario(
        test_chain=go_from_homepage_to_book_call_page_with_the_cta(driver, logger)
    ),
)

```

drive_page означает, что мы берём под контроль *одну* страницу.

Каждый *переход* выражается явно — открытием нового **drive_page**.

Внутри **act** выражает действие, совершаемое на этой странице: это *test step*. **drive_page** вариативен: он принимает сколько угодно вызовов **act**, а запятая между ними превращается в AND:

Открыть, затем проверить домашнюю страницу. И кликнуть по СТА. Меняем страницу: проверить, что мы на странице book-a-call.

Иммунная система

Попробуем вызвать **verify_book_call_page** на **homepage**:

```

act(on_homepage, verify_book_call_page)

# error: Argument 2 to "act" has incompatible type
# "Callable[[BookCallPage], BookCallPage]";
# expected "Callable[[Homepage], Homepage]"

```

Действие несовместимо со своей целью. Эта программа не *компилируется*.

Забудем про **.success**:

```

drive_page(
    act(on_book_a_call_page, verify_book_call_page)
    .failure(just_log_error("Failed to verify the 'Book a call' page..."))
)

# error: Expected type 'ActionSuccess[TPOM ≤: POMBase]',
# got 'ActionFailure[BookCallPage]' instead

```

Поставим **.success** сразу после **act**:

```

drive_page(
    act(on_book_a_call_page, verify_book_call_page)
    .success(
        log_success_with_current_url_and_take_screenshot(
            "On the 'Book a call' page!"
        )
    ),
),

# error:
# "ActionStart[BookCallPage]" has no attribute "success"

```

```
# Unresolved attribute reference 'success' for class 'ActionStart'
```

Поменяем местами `.success` и `.failure`:

```
drive_page(
    act(on_book_a_call_page, verify_book_call_page)
    .success(
        log_success_with_current_url_and_take_screenshot(
            "On the 'Book a call' page!"
        )
    )
    .failure(just_log_error("Failed to verify the 'Book a call' page...")),
),

# error:
# "ActionStart[BookCallPage]" has no attribute "success"
# Unresolved attribute reference 'success' for class 'ActionStart'
```

Сцепим разнородные вызовы `act` внутри одного `drive_page`:

```
drive_page(
    act(on_homepage, open_then_verify_homepage)
    .failure(just_log_error("Failed to reach the homepage..."))
    .success(
        log_success_with_current_url_and_take_screenshot(
            "On the homepage!"
        )
    ),
    act(on_homepage, click_book_call_page_cta)
    .failure(just_log_error("Failed to click on the 'Book a call' CTA..."))
    .success(just_log_success("Clicked on the 'Book a call' CTA!")),
    act(on_book_a_call_page, verify_book_call_page) # <- [!]
    .failure(just_log_error("Failed to verify the 'Book a call' page..."))
    .success(
        log_success_with_current_url_and_take_screenshot(
            "On the 'Book a call' page!"
        )
    ),
),

# error: Expected type 'ActionSuccess[Homepage]'
# (matched generic type 'ActionSuccess[TPOM ≤: POMBase]'),
# got 'ActionSuccess[BookCallPage]' instead
```

Многие команды грызутся из-за *coding styles*.

Osagına не церемонится: если стиль не соблюден — это не *lint*-ошибка и не предупреждение. Оно просто не **компилируется**.

Osagına навязывает один и тот же шаблон всем, и эти ошибки всплывают прямо в редакторе — через *туру*: мгновенная обратная связь.

Для тестовых сценариев в приоритете — однородность и простота. Только и всего.

match_page

`match_page` обрабатывает ситуации, когда страница может отрисовываться по-разному.

Начнём с принципа *matcher*'ов:

```
@final
class PageWithCookiesBannerMatchers:
    """Drive nuts anybody with this page or use matchers."""

    def __init__(self, *, driver: WebDriver) -> None:
        """Initialize helper."""
        self._driver = driver

    def has_cookies_banner(self) -> bool:
        """Quickly verify if the cookies banner is displayed."""
        timeout = min(get_timeout(), 5)

        try:
            WebDriverWait(self._driver, timeout).until(
                ec.visibility_of_element_located(
                    (By.CSS_SELECTOR, '[data-testid="cookies-banner"]')
                )
            )
        except TimeoutException:
            return False
        return True

    def has_not_cookies_banner(self) -> bool:
        """Quickly verify if the cookies banner is NOT displayed."""
        timeout = min(get_timeout(), 5)

        try:
            WebDriverWait(self._driver, timeout).until(
                ec.invisibility_of_element_located(
                    (By.CSS_SELECTOR, '[data-testid="cookies-banner"]')
                )
            )
        except TimeoutException:
            return False
        return True
```

Matcher выполняет минимальную проверку того, истинно ли нечто, — и максимально быстро.

 И всё же не хватайтесь за голый `.find_element(s)`: это прямая дорога к *Selenium-flakiness*.

Ограничение в 5 секунд почти не сказывается на горизонтально масштабируемой батарее тестов: тут об этом беспокоиться незачем. Не стоит и выдавать `verify` за *matcher*: **это два разных инструмента**.

Применение в сценарии:

```
on_homepage = Homepage(driver=driver)
check_that_page = PageWithCookiesBannerMatchers(driver=driver)

# * ...
[
    match_page(
```

```

branches=[
  when(
    check_that_page.has_cookies_banner,
    name="Has cookies banner",
    then=[
      drive_page(
        act(on_homepage, confirm_cookie_banner)
        .failure(
          log_error_with_current_url(
            "Failed to click on the cookies banner's confirm button..."
          )
        )
        .success(
          log_success_with_current_url_and_take_screenshot(
            "Clicked on the cookies banner's confirm button!"
          )
        )
      )
    ],
  ),
  when(
    check_that_page.has_not_cookies_banner,
    name="Has NOT cookies banner",
    then=[],
  ),
],
logger=create_matching_logger("terminal"),
# <- [!] If you want debug logs
),
drive_page(
  act(on_homepage, ...)
  .failure(...)
  .success(...)
),
]

```

`match_page` стоит на одном уровне с `drive_page` и точно так же сцепляется в цепочки. Его команда `then` ждёт цепочку вызовов `drive_page` или `match_page`. Ветви задаются через `when`.

`match_page` и `when` появились в Oscarina поздно: Igoristan оказался настолько непредсказуемым, что необходимость в них стала очевидной.

Их интеграция прошла легко — доказательство гибкости грамматики: вполне могут появиться и другие подобные структуры.

Повторения

Чтобы повторить тестовую цепочку (например, чтобы проверить несколько попыток несанкционированного доступа), просто умножьте список:

```

[
  drive_page(
    act(on_dashboard_welcome_page, click_on_go_to_nested_page_btn)
    .failure(
      just_log_error("Failed to click on the go-to-nested-page button...")
    )
    .success(just_log_success("Clicked on the go-to-nested-page button!")),

```

```

        act(on_dashboard_welcome_page, verify_missing_otp_msg_is_displayed)
        .failure(
            just_log_error(
                "Failed to find the missing OTP auth message...",
            )
        )
        .success(
            log_success_with_current_url_and_take_screenshot(
                "Found the missing OTP auth message!"
            )
        ),
    ),
] * 5 # <- [!]

```

Фрагменты

Фрагмент — это функция `(driver, logger) -> TestChain`, которую можно внедрить до или после основной цепочки — через `pre_test_scenarios_fragments` и `post_test_scenarios_fragments`.

Например, `login_without_otp_happy_path` — это фрагмент:

```

def login_without_otp_happy_path(driver: WebDriver, logger: ILogger):
    """Verify that we can connect without OTP."""
    on_dashboard_login_page = DashboardLoginPage(driver=driver)
    on_dashboard_welcome_page = DashboardWelcomePage(driver=driver)

    # * ...
    return [
        drive_page(
            act(on_dashboard_login_page, open_dashboard_login_page)
            .failure(just_log_error("Failed to open the dashboard login page..."))
            .success(just_log_success("Opened the dashboard login page!")),
            # * ...
        ),
        # * ...
    ]

```

Внедрение в начале:

```

test_cant_access_the_protected_page_without_otp_using_the_ui = create_selenium_test(
    name="Can't access the protected page without OTP (using the UI)",
    test_scenario=lambda driver, logger: Scenario(
        test_chain=dashboard_access_to_protected_page_without_otp_using_the_ui(
            driver, logger
        )
    ),
    pre_test_scenarios_fragments=[login_without_otp_happy_path], # <- [!]
)

```

Внедрение в конце:

```

test_dashboard_login_page_back_to_igoristan_button = create_selenium_test(
    name="Use the go back to Igoristan button",

```

```

test_scenario=lambda driver, logger: Scenario(
    test_chain=just_go_back_to_igoristan(driver, logger)
),
post_test_scenarios_fragments=[verify_homepage], # <- [!]
)

```

Оба параметра можно комбинировать, и каждый принимает список *фрагментов*, которые внедряются в заданном порядке.

Псевдонимы

Сценарии могут становиться громоздкими.

Поскольку всё декларативно, пользователь волен заводить псевдонимы:

```

on_homepage = Homepage(driver=driver)
check_that_page = PageWithCookiesBannerMatchers(driver=driver)

click_confirm_cookies = drive_page(
    act(on_homepage, confirm_cookie_banner)
    .failure(
        log_error_with_current_url(
            "Failed to click on the cookies banner's confirm button..."
        )
    )
    .success(
        log_success_with_current_url_and_take_screenshot(
            "Clicked on the cookies banner's confirm button!"
        )
    )
)

# * ...
[
    match_page(
        branches=[
            when(
                check_that_page.has_cookies_banner,
                name="Has cookies banner",
                then=[click_confirm_cookies], # <- [!]
            ),
            when(
                check_that_page.has_not_cookies_banner,
                name="Has NOT cookies banner",
                then=[],
            ),
        ],
        logger=create_matching_logger("terminal"),
        # <- [!] If you want debug logs
    ),
    drive_page(
        act(on_homepage, ...)
        .failure(...)
        .success(...)
    ),
]

```

Любому значению можно дать псевдоним и переиспользовать его.
Такая запись чистая: немедленного *эффекта* она не производит.
Всё можно переобъявить и перекомпоновать в другом месте — лишь бы итоговая цепочка соответствовала ожидаемой.

Первые дзюцу

И подо льдом поблёскивает река...

Наборы данных

Гонять тест через набор данных в Oscarina просто:

```
multi_login_dataset: Sequence[MappingProxyType[ImmutableCredentialsKeys, str]] = [
    MappingProxyType(
        {
            "login": "any",
            "password": "figatellu",
        }
    ),
    MappingProxyType(
        {
            "login": "Napoleon",
            "password": "figatellu",
        }
    ),
    MappingProxyType(
        {
            "login": "NoSicilianAllowed",
            "password": "figatellu",
        }
    ),
    MappingProxyType(
        {
            "login": "anonymous",
            "password": "figatellu",
        }
    ),
    MappingProxyType(
        {
            "login": "TheEmpire",
            "password": "figatellu",
        }
    ),
],

def _create_login_scenario(credentials: ImmutableCredentials) -> SeleniumTestScenario:
    """Welcome to functional factories."""

    def _scenario(driver: WebDriver, logger: ILogger):
        dashboard_creds = credentials # <- [!] Provided by the closure

        on_dashboard_login_page = DashboardLoginPage(driver=driver)
        on_dashboard_welcome_page = DashboardWelcomePage(driver=driver)

        # * ...
        return Scenario(
            test_chain=[
```



```

        drive_page(
            # * ...
            act(
                on_dashboard_login_page,
                login_without_otp_and_with_retries(
                    dashboard_creds, # <- [!]
                    retries_amount,
                    logger=logger,
                ),
            ),
            .failure(
                just_log_error(
                    "Failed to connect to the dashboard without OTP...",
                )
            ),
            .success(
                just_log_success(
                    f"Connected to the dashboard as {dashboard_creds['login']}!"
                    # 📈 [!]
                )
            ),
        ),
        drive_page(
            act(on_dashboard_welcome_page, ...)
            .failure(...)
            .success(...)
        ),
    ]
)

return _scenario

multi_login_tests = [
    create_selenium_test(
        name=f"Login - {creds['login']}",
        test_scenario=_create_login_scenario(creds),
    )
    for creds in multi_login_dataset
]

```

Достаточно одного [closure](#)^[1].

Обратите внимание: **Scenario** здесь объявляется изнутри. И это логично — вся суть как раз в том, чтобы её инкапсулировать.

Итак, **multi_login_tests** — это *list* объектов **Test**, который мы *unpack*'аем в **TestSuite**, вот так:

```

def create_suite(
    *,
    drivers_pool: SeleniumWebDriversPool,
) -> TestSuite:
    return TestSuite(
        name="Login (data-driven PoC)",
        tests=[*multi_login_tests],
        drivers_pool=drivers_pool,
    )

```

Smoke-тесты

Чтобы запустить smoke-тесты в начале цикла на Oscarina:

```
E2E_CYCLE_NAME = "My very first cycle with Oscarina"

def create_e2e_test_cycle(drivers_pool: SeleniumWebDriversPool):
    """e2e test cycle."""
    return TestCycle(
        name=E2E_CYCLE_NAME,
        campaigns=[
            create_my_first_campaign(drivers_pool=drivers_pool),
        ],
        smoke_tests_campaigns=[
            create_my_first_smoke_campaign(drivers_pool=drivers_pool),
            create_my_second_smoke_campaign(drivers_pool=drivers_pool),
        ],
        mode="wait-for-all-smoke-tests",
    )
```

`mode` принимает два значения (по умолчанию:

`"fail-fast-on-first-smoke-campaigns-sequence-fail")`:

- `"fail-fast-on-first-smoke-campaigns-sequence-fail"`: как только проваливается одна кампания smoke-тестов, остальные пропускаются.
- `"wait-for-all-smoke-tests"`: все кампании smoke-тестов обрабатывают до конца, даже если какая-то по пути провалилась.

В обоих случаях основные тесты пропускаются, если провалился хоть один smoke-тест.

Setup и teardown

`Scenario` принимает два опциональных обратных вызова: `setup` и `teardown`.

```
Scenario(
    setup=seed_test_user,
    test_chain=[
        drive_page(
            act(page, open_page)
            .failure(log_error("Failed to open..."))
            .success(log_success("Opened!")),
        ),
    ],
    teardown=delete_test_user,
)
```

Жизненный цикл

1. `setup()`: выполняется перед `test_chain`. При сбое `test_chain` пропускается, а `teardown` всё равно выполняется. Если все попытки провалились из-за `setup`, тест помечается как **SKIPPED** (а не **FAILED**);
2. `test_chain`: собственно шаги теста;
3. `teardown()`: выполняется всегда, даже при сбое. Ошибки логируются и игнорируются.

`setup` и `teardown` — это `Effect`.

Они предназначены для инфраструктурных задач: наполнить базу данных, вызвать API, подчистить состояние...

Если нужна инкапсуляция: *closure*.

Паттерн Проху

Один из примеров применения из [канонического примера](#)^[2] — `HumanizedDriver`:

```
class HumanizedDriver(WebDriver):
    def __init__(
        self, driver: WebDriver, **keyboard_config: Unpack[KeyboardConfig]
    ) -> None:
        object.__init__(self)
        self._driver = driver
        self._config = keyboard_config

    def find_element(
        self,
        by: str | RelativeBy = "id",
        value: str | None = None,
    ) -> _HumanizedWebElement:
        element = self._driver.find_element(by, value)
        return _HumanizedWebElement(element, self._config)

    def find_elements(
        self,
        by: str | RelativeBy = "id",
        value: str | None = None,
    ) -> list[WebElement]:
        elements = self._driver.find_elements(by, value)
        return [_HumanizedWebElement(el, self._config) for el in elements]

    def __getattr__(self, name: str):
        return getattr(self._driver, name)
```

Идея: возвращать *Web Elements*, которые ведут себя иначе при взаимодействиях, похожих на пользовательские. В нашем случае — нажатия клавиш.

Прозрачно для *type system*, прозрачно для *runtime*.

Что затем позволяет:

```
create_selenium_test(
    name="Send the form",
    test_scenario=lambda driver, logger: Scenario(
        test_chain=_send__form(
            HumanizedDriver( # <- [!]
                driver,
                wpm=125,
                typo_rate=0.14,
                hesitation_rate=0.02,
                burst_rate=0.35,
                late_correction_rate=0.6,
            ),
            logger,
        ),
    ),
```

```
),  
)
```

Или, с *closure*:

```
def _scenario(driver: WebDriver, logger: ILogger):  
    humanized_driver = HumanizedDriver( # <- [!]  
        driver,  
        wpm=125,  
        typo_rate=0.14,  
        hesitation_rate=0.02,  
        burst_rate=0.35,  
        late_correction_rate=0.6,  
    ),  
  
    on_some_form_page = SomeFormPage(driver=humanized_driver) # <- [!]
```

Тот же принцип работает и для логгера — например, перенаправляя его в *sink*. Канонически Oscarina этот случай не покрывает.

Реактивное программирование: НЕТ

Тестовые сценарии Oscarina намеренно статичны.

Однако веб-приложение динамично, и порой вполне законно перехватить значение на лету и передать его на более поздний шаг.

Oscarina на это не отвечает. Ей это и не нужно.

Архитектурный ответ

Здесь нам нужен *in-memory cache*.

Мы генерируем ключи прямо перед стартом тестовой цепочки и передаём их в действия POM. Действия пишут и читают по уникальному ключу.

Сценарий лишь раздаёт их:

```
# * ...  
cache = in_memory_cache_with_30m_ttl  
username_key = reserve_free_cache_key(cache)  
otp_send_date_key = reserve_free_cache_key(cache)  
  
return [  
    drive_page(  
        # * ...  
        act(  
            on_dashboard_login_page,  
            start_to_login_with_otp_and_with_retries(  
                dashboard_creds,  
                retries_amount,  
                cache=cache,  
                logger=logger,  
                username_key=username_key,  
                otp_send_date_key=otp_send_date_key,  
            ),  
        ),  
    ]
```

```

        .failure(
            just_log_error(
                "Failed to fill and confirm the login form with OTP...",
            )
        )
        .success(
            just_log_success(
                "Filled and confirmed the login form with OTP!"
            )
        ),
        act(
            on_dashboard_login_page,
            verify_otp_screen,
        )
        .failure(
            just_log_error(
                "Failed to verify the OTP screen...",
            )
        )
        .success(just_log_success("Verified the OTP screen!")),
        act(
            on_dashboard_login_page,
            type_otp_with_retries(
                retries_amount,
                cache=cache,
                logger=logger,
                username_key=username_key,
                otp_send_date_key=otp_send_date_key,
            ),
        )
        .failure(
            just_log_error(
                "Failed to confirm the OTP code...",
            )
        )
        .success(just_log_success("Confirmed the OTP code!")),
    ),
    # * ...
]

```

Вызовы API и блокировки

API и блокировки нужно обрабатывать в POM-ах.

 *Ocarina не поддерживает `async/await` и никогда не будет.*

Вызовы API: достаточно синхронного `requests`.

Блокировки: `threading.Lock`, если в каждый момент работает только один процесс; иначе хватит распределённых блокировок Redis (`redis.StrictRedis + redis.lock`).

Профиль браузера

Некоторые случаи требуют передать профиль через `--profile-path`:

- Аутентификация на прокси,
- Предустановленные расширения,

- **Локальные настройки** (язык, часовой пояс, сертификаты...),
- и так далее.

[1] [https://en.wikipedia.org/wiki/Closure_\(computer_programming\)](https://en.wikipedia.org/wiki/Closure_(computer_programming))

[2] <https://github.com/mojo-molotov/ocarina-example>

Первые реальные препятствия

Единственным разумным решением было приспособиться к существующим условиям.

Случайные ошибки сервера

Случайные ошибки сервера — крепкий орешек.

На одной особенно неприятной работе мне пришлось иметь дело с окружением, которое регулярно выдавало совершенно случайные страницы ошибки 500 — в какой бы раздел приложения ты ни зашёл.

В таких случаях ответ Oscarina кроется прямо в том, как создаётся глагол `act`:

```
ERROR_PAGE_REGEX = re.compile(r"^\d{3}(?!\d)")

@final
class HttpErrorPageReachedError(Exception):
    """Raised when error page is reached."""

def act(pom: TPOM, action: Callable[[TPOM], TPOM]) -> ActionStart[TPOM]:
    """Act on a page."""

    def failure_hook(pom: TPOM, exc: Exception) -> Fail:
        with suppress(Exception):
            title = pom.get_current_title()
            is_http_error_page = title and ERROR_PAGE_REGEX.match(title.strip())
            if is_http_error_page:
                http_error = HttpErrorPageReachedError(f"HTTP error page: {title}")
                http_error.__cause__ = exc
                return Fail(error=http_error)
        return Fail(error=exc)

    return create_act(
        pom,
        action,
        on_failure=failure_hook, # <- [!]
    )
```

Хук `on_failure` создан именно для этого.

Достаточно завести несколько *предохранителей* и подменить ошибку, обёрнутую в `Fail`, — чтобы запустить *повторный прогон* любого теста, упавшего по внешней причине.

Следующий шаг должен быть вам знаком:

```
transient_errors = (
    HttpErrorPageReachedError, # <- [!]
    PageVerificationError,
    # * ...
)

@final
```

```

class TestSuite(OriginalTestSuite[WebDriver]):
    """TestSuite adapter."""

    def __init__(
        self,
        *,
        # * ...
    ) -> None:
        """Initialize the TestSuite."""
        # * ...
        super().__init__(
            # * ...
            max_retries_per_test=8,
            transient_errors=transient_errors,
        )

```

Наконец, если в проекте используется ещё и `match_page`, а общая переменная `transient_errors` нежелательна, не забудьте добавить эти новые классы ошибок в параметр `raised_exceptions` конструктора `match_page`.

Логи автотестов станут хорошей отправной точкой, чтобы поднять эти проблемы перед командой.

Случайные ошибки в пределах шага

Решив, что вся эта дичь осталась позади, я двинулся дальше — и тут же наткнулся на нестабильные формы и системы аутентификации, которые работали через раз.

Здесь ответ Oscarina иной: ответственность мы передаём POM-у.

```

@final
class CorsicamonEnterXXXKeyPage(SeleniumTitleMixin, POMBase):
    """Igoristan's corsicamon enter XXX key page."""

    def enter_xxx_key(
        self
    ) -> CorsicamonEnterXXXKeyPage:
        """Enter XXX key."""
        # * ...

    # * ...
    def enter_xxx_key_with_retries(
        self, *, retries: int, logger: ILogger
    ) -> CorsicamonEnterXXXKeyPage:
        """Enter XXX key (n retries)."""
        validate(retries, name="retries").assert_that(
            is_positive
        ).execute().raise_if_invalid()

        attempts_count = 1
        self.enter_xxx_key()

        while attempts_count <= retries:
            timeout = get_timeout()
            with suppress(Exception):
                WebDriverWait(self._driver, timeout).until(
                    ec.invisibility_of_element_located(

```



```

        self._corsicamon_network_error_container
    )
)
break

msg = (
    "Failed to enter the XXX Key."
    "\n"
    f"Life: {attempts_count}/{retries}"
    "\n"
    f"Current URL: {self._driver.current_url}"
)

logger.warning(msg)
take_screenshot(driver=self._driver, logger=logger, category="WARNING")
self.click_retry_button()
attempts_count += 1

s = "s" if attempts_count > 1 else ""
msg = f"Entered the XXX Key. After {attempts_count} attempt{s}."

logger.info(msg)
return self

```

Тут же встаёт вопрос о *connectors*: как передавать в них параметры?

```

"""Functional connectors."""

# * ...

def enter_xxx_key(
    p: CorsicamonEnterXXXKeyPage,
) -> CorsicamonEnterXXXKeyPage:
    """Enter the XXX key."""
    return p.enter_xxx_key()

# * ...

def enter_xxx_key_with_retries(
    *,
    retries: int,
    logger: ILogger,
) -> Callable[[CorsicamonEnterXXXKeyPage], CorsicamonEnterXXXKeyPage]:
    """Click on the retry button."""

    def unwrapped(p: CorsicamonEnterXXXKeyPage) -> CorsicamonEnterXXXKeyPage:
        return p.enter_xxx_key_with_retries(retries=retries, logger=logger)

    return unwrapped

```

Просто верните **def** с нужной сигнатурой — изнутри функции, которая захватывает параметры. Это и есть *closure*^[1].

Случайные ошибки Selenium

Selenium даёт массу возможностей выстрелить себе в ногу: *race conditions*, ошибки *stale element* и так далее.

Ответ здесь **прагматичный**: добавьте `WebDriverException` прямо в `transient_errors` — со щедрым числом повторов (8, то есть 9 жизней, как у кошки 🐱).

Перехватывайте все ошибки Selenium и следите за повторами в логах.

После этого легко вычислить тесты, которые не помешало бы доработать.

Дискретные случайные ошибки

Ещё удивительнее: приложения, которые без видимой причины показывают тосты с ошибками, или формы, которые ругаются ошибками валидации на совершенно корректный ввод — при этом реально ничего не блокируя.

Эти ошибки поймать труднее всего — именно потому, что они *безболезненны*. Тут не получится просто увидеть падение и навесить *политику повторов*, пока баг не починят. По сути, они невидимы.

Что остаётся? Кромсать тестовые сценарии или хвататься за «ниндзя-техники». Osagima отвергает и то, и другое.

Используйте *watchers*:

```
def catch_me_if_you_can_cb(watcher: SeleniumWatcher) -> None:
    """Detect any element with CSS class 'catch-me-if-you-can' on the current page."""
    # NOTE: using JS here to bypass the implicit wait timeout.
    elements = watcher.driver.execute_script(
        "return Array.from(document.querySelectorAll('.catch-me-if-you-can'));"
    )

    if not elements:
        return

    raw = watcher.driver.execute_script(
        """
        return arguments[0].map(el => ({
            tag:      el.tagName.toLowerCase(),
            text:     el.innerText.trim(),
            id:       el.id,
            cls:      el.className,
            name:     el.getAttribute('name') || '',
            testid:   el.getAttribute('data-testid') || '',
        }));
        """,
        elements,
    )

    for attrs in raw:
        fingerprint = ":".join(
            filter(
                None,
                [
                    attrs["tag"],
                    attrs["text"],
                    attrs["id"],
```

```

        attrs["cls"],
        attrs["name"],
        attrs["testid"],
    ],
)
)

if fingerprint in watcher.cache:
    continue

watcher.cache.add(fingerprint)
watcher.report(
    f"catch-me-if-you-can element detected: <{attrs['tag']}> {attrs['text']!r}",
    label="CATCH_ME_IF_YOU_CAN",
)

# * ...

test_send_chaotic_form = create_selenium_test(
    name="Send the chaotic form",
    test_scenario=lambda driver, logger: Scenario(
        test_chain=_send_chaotic_form(
            HumanizedDriver(
                driver,
                wpm=125,
                typo_rate=0.14,
                hesitation_rate=0.02,
                burst_rate=0.35,
                late_correction_rate=0.6,
            ),
            logger,
        ),
        watchers=[ # <- [!]
            create_selenium_watcher(
                callback=catch_me_if_you_can_cb,
                name="catch-me-if-you-can",
                poll_interval=0.8,
            ),
        ],
    ),
)
)

```

`catch_me_if_you_can_cb` — это *callback*, который *watcher* вызывает каждые 0,8 секунды (`poll_interval`).

Уточним несколько моментов.

Использование JS

Watcher устойчив к ошибкам: исключения он молча проглатывает.

Поэтому нет смысла брать функцию Selenium, чтобы достать элемент страницы, — это лишь добавило бы лишнего балласта.

А родные функции Selenium заставили бы возиться с *implicit timeout*.

Если идти напрямую через *Javascript*, это обходит всю внутреннюю логику *polling* и делает работу *watcher* максимально неблокирующей для теста, который выполняется на том же *driver*.

Весь трюк остаётся незаметным — он занимает всего несколько миллисекунд.

Отпечатки

Watchers предоставляют простой строковый кэш, созданный как раз под эту задачу: если одна и та же ошибка остаётся на виду и фиксируется каждые 0,8 секунды, нет смысла снова и снова наткнуться на неё в скриншотах, отчётах и логах. Отпечаток позволяет пропускать то, что вы уже видели.

Отчёт

В конце *callback* вызывает `watcher.report`.

Этот вызов берёт на себя:

1. запись в лог того трения, которое обнаружил *watcher*;
2. создание скриншота — как следа обнаруженного.

HumanizedDriver

Ничто не мешает нам навешивать поведение на *logger* или *driver*. Здесь, раз уж форма капризная, мы выбираем медленный, «очеловеченный» тест: ввод текста с опечатками, исправлениями и заминками. Мы просто оборачиваем *driver* в *proxy* — `HumanizedDriver`.

Гейзенбаги конкурентности

Мои поиски на этом не закончились.

Как-то раз, прямо там, в офисе, я наблюдал, как коллеги считают до трёх, чтобы потом разом кликнуть и запустить одно и то же действие. Я поймал себя на мыслях о смысле собственной жизни. И всё же таким способом они и правда умудрялись воспроизводить баги.

Это поведение можно воспроизвести и средствами *Oscarina*.

По умолчанию *Oscarina* агрессивна.

Её опция `saturate_workers` принудительно и случайным образом клонирует тесты внутри набора.

Всякий раз, когда в *DriversPool* доступно больше *workers*, чем тестов для прогона в наборе, *Oscarina* случайным образом наклонирует тесты, поднимет все драйверы и раздаст каждому по тесту.

Эту опцию можно включить из функции `bootstrap`. Её также можно переключать точно — на уровне набора или кампании.

При конфликте последнее слово за самым глубоким элементом иерархии.

Например, если кампания опцию отключает, а набор включает, приоритет за набором.

```
if __name__ == "__main__":
    with timing(prefix="Tests duration:"):
        bootstrap(
            saturate_workers=False, # <- True by default
            # * ...
        )

# * ...
def create_campaign(
    *, drivers_pool: SeleniumWebDriversPool
) -> TestCampaign:
    return TestCampaign(
        saturate_workers=True, # <- 'None' by default (cascade)
```

```

    max_workers=16, # <- 'None' by default (CLI value)
    # 📌 Forcing saturate workers policy and 16 workers on this
    # campaign.
    # * ...
)

# * ...
def create_suite(
    *,
    drivers_pool: SeleniumWebDriversPool,
) -> TestSuite:
    return TestSuite(
        saturate_workers=False, # <- 'None' by default (cascade)
        # 📌 Will take the priority: saturate workers disabled on this
        # suite.
        # * ...
    )

```

Можно также временно собрать набор из одного-единственного теста — чтобы добиться предельной точности нацеливания.

Или запустить цикл сразу на нескольких машинах (горизонтальное масштабирование).

Помимо вопросов конкурентности, этот механизм клонирования призван ещё и гарантировать, что проходящие тесты ничем не обязаны случайности. И эффект тем сильнее, чем выше степень горизонтального масштабирования и чем больше задействовано workers.

[1] [https://en.wikipedia.org/wiki/Closure_\(computer_programming\)](https://en.wikipedia.org/wiki/Closure_(computer_programming))

Расширяемость

Если мы не верим в свободу слова для тех, кого презираем, значит, не верим в неё вовсе.

ValidationChain

`validate` применим внутри POM-ов и позволяет описывать инварианты в виде цепочек.

Выполнение **отложенное**: `.execute()` нужно вызвать явно.

Результат предоставляет `is_valid`, `errors` и `validated_values`. По умолчанию он инертен. При необходимости исключение выбросит `.raise_if_invalid()`.

```
validate(checkbox.is_selected(), name="checkbox_is_selected").assert_that(
    is_truthy, msg="Couldn't select the OTP checkbox.")
).execute().raise_if_invalid()
```

Сцепление инвариантов

Несколько утверждений для одного значения:

```
validate(unsafe_min_date, name="cached_min_date")
.assert_that(is_str)
.assert_that(is_iso_utc_date_string).execute().raise_if_invalid()
```

Сцепление валидаций

Несколько валидаций для разных значений:

```
chain_validations(
    validate(unsafe_username, name="cached_username").assert_that(is_str),
    validate(unsafe_min_date, name="cached_min_date")
    .assert_that(is_str)
    .assert_that(is_iso_utc_date_string),
).execute().raise_if_invalid()
```

Переиспользуемые инварианты

Чтобы вынести повторяющуюся валидацию, создайте *Invariant Validator*:

```
def _workers_amount_chain(
    chain: ValidationStartBlock[int],
    value: int,
) -> ValidationAssertBlock[int]:
    msg = f"Value Error: Number of workers must be at least 1 (got: {value})."
    return chain.assert_that(is_positive, msg=msg).assert_that(is_not_zero, msg=msg)

def validate_workers_amount(
    *, workers_amount: int, name: str
```

```

) -> ValidationAssertBlock[int]:
    """Validate that workers amount is at least 1."""
    return FrameworkInvariantValidator.create(
        workers_amount, name, _workers_amount_chain
    )

# * ...
validate_workers_amount(
    workers_amount=max_workers, name="max_workers"
).execute().raise_if_invalid()

```

Соглашение: `FrameworkInvariantValidator.create` — для технических инвариантов, `BusinessInvariantValidator.create` — для бизнес-инвариантов.

Пользовательские утверждения

Без аргумента:

```

def is_str(value: Any) -> None:
    if not isinstance(value, str):
        msg = "Expected value to be string."
        raise InvariantViolationError(msg)

```

С аргументом:

```

def is_equal_to(cmp: Any) -> Predicate[Any]:
    def unwrapped(value: Any) -> None:
        if value != cmp:
            msg = f"{value} is not equal to {cmp}."
            raise InvariantViolationError(msg)

    return unwrapped

```

Безопасность типов

Тайпчекер ловит утверждения, несовместимые с типом значения:

```

validate("lol", name="n").assert_that(is_positive)

# error: Argument 1 to "assert_that" of "ValidationStartBlock" has
# incompatible type "Callable[[float], None]";
# expected "Callable[[str], None]"

```

Success и Failure

`.success` и `.failure` — каждый принимает *effect*, который нужно выполнить.

Канонический пример^[1] реализует несколько обработчиков: простое логирование ошибок, логирование ошибок с текущим URL, логирование успеха и логирование успеха со скриншотом (+ URL).

```

def _append_current_url_in_msg(msg: str, driver: WebDriver) -> str:
    try:

```

```

        driver_healthcheck(driver)
        extended_msg = f"{msg}\nCurrent URL: {driver.current_url}"
    except DriverDiedError:
        extended_msg = f"{msg}\nThe WebDriver is down, can't provide the current URL."

    return extended_msg

def create_just_log_error(*, logger: ILogger) -> Callable[[str], FailureHandler]:
    return lambda msg: lambda exc: logger.error(msg, exc=exc)

def create_log_error_with_current_url(
    *, logger: ILogger, driver: WebDriver
) -> Callable[[str], FailureHandler]:
    def unwrapped(msg: str) -> FailureHandler:
        def _log_error_with_url_effect(exc: Exception) -> None:
            extended_msg = _append_current_url_in_msg(msg, driver)
            return create_just_log_error(logger=logger)(extended_msg)(exc)

        return _log_error_with_url_effect

    return unwrapped

def create_just_log_success(*, logger: ILogger) -> Callable[[str], SuccHandler]:
    def unwrapped(msg: str) -> SuccHandler:
        def _log_effect() -> None:
            logger.success(msg)

        return _log_effect

    return unwrapped

def create_log_success_and_take_screenshot(
    *, logger: ILogger, driver: WebDriver
) -> Callable[[str], SuccHandler]:
    def unwrapped(msg: str) -> SuccHandler:
        def _log_and_take_screenshot_effect() -> None:
            performed_dependent_effect = create_just_log_success(logger=logger)(msg)()
            take_screenshot(driver=driver, logger=logger, category="SUCCESS")
            return performed_dependent_effect

        return _log_and_take_screenshot_effect

    return unwrapped

def create_log_success_with_current_url_and_take_screenshot(
    *, logger: ILogger, driver: WebDriver
) -> Callable[[str], SuccHandler]:
    def unwrapped(msg: str) -> SuccHandler:
        def _log_success_with_url_and_take_screenshot_effect() -> None:
            return create_log_success_and_take_screenshot(logger=logger, driver=driver)(
                _append_current_url_in_msg(msg, driver)
            )()

        return _log_success_with_url_and_take_screenshot_effect

```



```
return unwrapped
```

Стоит подумать и о других обработчиках:

- `create_log_error_with_retry_hint`: сигнализирует о *transient error*, а значит — о возможной flakiness;
- `create_log_error_and_send_alert`: при неудаче отправляет webhook, не засоряя сам тест;
- `create_log_success_and_record_timing`: фиксирует метку времени завершения, чтобы измерить фактическую длительность шага (его комбинируют с `on_run_effect` из `create_act`);
- и так далее.

Стоит подумать и о *combinator*'е.

Плагины

`bootstrap` позволяет запускать плагины постобработки, опираясь на результаты тестового цикла. Например, `generate_docx_proof` проходит по дереву логов и генерирует по одному документу Word (тестовому доказательству) на каждый тестовый случай, встраивая скриншоты и переводя метки времени UTC в локальное время.

Идея: плагины пересобирают артефакты, накопленные по ходу дела, в другую форму. Например, плагин, генерирующий отчёт в виде веб-дашборда, напрашивается сам собой.

Расширяемая грамматика

Грамматика тестовых сценариев построена на одном-единственном типе — `ChainRunner[T]`. Сценарий — это `list[ChainRunner]`, который выполняется последовательно и обрывается на первом же сбое. `drive_page` — это просто тонкая обёртка вокруг `chain_actions`, которая строит `ChainRunner`. Любая функция, возвращающая `ChainRunner`, подключается, не затрагивая сам фреймворк.

`match_page` добавили позже — чтобы обрабатывать страницы с переменным состоянием (опциональные баннеры, A/B-тесты, страницы техобслуживания...): он проверяет условия по порядку и запускает первую подошедшую ветвь.

Ещё один пример — `skip_if`: намеренный пропуск части сценария по условию, без сбоя (вернул бы нейтральный `Ok`); пригодится для необязательных шагов, зависящих от окружения или данных.

Единственный контракт расширения: верните `ChainRunner`.

[1] <https://github.com/mojo-molotov/ocarina-example>

Использование Ocarina с AI

Привет, это я, твой новый лучший друг!

Рабочая конфигурация: полный цикл тестирования, собранный связкой Claude Code и Ocarina, против публичного демо Katalon CURA.

 [Возьмите за образец AI-пример.](#)^[1]

Три духовных камня

1. **CLAUDE.md** в корне проекта.
2. **skills/** с одним **<name>/SKILL.md** на каждую процедуру.
3. Правило проверки: каждое утверждение о SUT идёт от наблюдения (probe, **gh api**, **curl -v**), а не от домысла.

CLAUDE.md

Два варианта. **CLAUDE.md** — полный (правила + структура проекта, иерархия, соглашения, устройство CI, шаблон PR). **CLAUDE.slim.md** — только правила. Slim — когда контекст перегружен; полный — для онбординга и ревью. При расхождении побеждает полный.

Шаги онбординга (venv, **pip install**, набор скиллов, скопированный в Claude Code, **ruff / мурп / pre-commit**, smoke-проверка раннера) описаны в **setup-environment**.

Правила:

Тестирование безопасности функциональное и статичное — никогда активное. Никаких пейлоадов, никаких сфабрикованных запросов, никаких манипуляций с DOM через DevTools. Сценарии чёрных шляп идут через обычный пользовательский интерфейс.

Используйте константы. Именованные значения не вписывают прямо в код.

Наборы данных — это решения людей. Предложить — не значит запустить.

Проверяйте поведение SUT эмпирически. Probe, **gh api** или **curl -v**. Никогда не домысел. Выводите заново каждый раз: probe отвечает только за то, что он реально прогнал; прежний диагноз — только для того прогона.

У каждого правила есть однострочное «почему».

skills/

Один файл Markdown на каждый навык, YAML-frontmatter + тело. Десять семейств.

Ревью (14)

Статическое чтение; выносят находки на поверхность.

- **review-spec-gaps** — уточняющие вопросы по FRD.
- **review-watcher-misuse** — **watcher.report(...)** вопреки соглашению «только негатив».

- `review-compartmentalisation-leaks` — URL, селекторы, магические числа не на своём месте.
- `review-dead-code` — неиспользуемые connectors / POM / сценарии / suites / фрагменты / константы; для каждой находки: удалить, инкубировать (`<source-root>/incubator/`, дерево зависимостей сохранено) или оставить.
- `review-hierarchy-naming` — родитель $\supset$ потомок с тем же именем в дереве цикла (чаще всего `Campaign("X")  $\supset$  Suite("X")`); переименовать потомка под его реальный сегмент (иерархия строгая — сплющивание невозможно).
- `review-report` — классифицировать каждый FAIL / SKIP за один прогон.
- Плюс: `review-type-ignore`, `review-match-candidates`, `review-unverified-transitions`, `review-submit-dispatchers`, `review-comment-drift`, `review-suite-stability`, `review-intent-collisions`, `review-watcher-emissions`.

Анализ (4)

- `analyse-flakiness` — расширить сеть transient-error; хронические падения — это настоящие flakes.
- `analyse-fixture-flakiness` — инструментировать setup/teardown; вынести на поверхность перекрёстное загрязнение тестов.
- `analyse-watcher-flakiness` — с каждым watcher и без него, перебор интервалов.
- `analyse-screenshot-flakiness` — сгруппировать по (`test`, `step`, `browser`), выявить различия.

Чёрная шляпа (6)

- `business-logic-vulnerability-ideation` — обрушить продукт.
- `incoherence-attack-ideation` — каждый шаг легален, набор невозможен.
- `persistence-attack-ideation` — упорные повторы заблокированных действий.
- `permission-appropriateness-audit` — уместна ли сама модель доступа?
- `bfcache-exposure-ideation` — атаки на BFCache.
- `lateral-resource-ideation` — IDOR только через адресную строку.

Понимание (4)

- `assess-test-base` — каталогизировать тестовую базу.
- `assess-ecosystem` — ограниченное исследование публичных источников, с потолком по бюджету токенов.
- `understand-sut-constraints` — ограничения SUT, которые ломают параллельные тесты.
- `understand-ocarina` — пройти по документации.

Выбор (3)

По mtime — никогда по имени файла.

- `pick-screenshots`, `pick-logs`, `pick-reports`.

Авторство (9)

Каждый выдаёт готовый результат.

- **empiricism** — проверяйте перед кодированием; не перезаписывайте intentional-fail гар-тесты.
- **write-a-probe** — одноразовый скрипт, в gitignore.
- **write-test-strategy** — сгенерировать документ test-strategy из набора (scope, types, таблицы покрытия, дерево цикла, pass/fail, gaps, CI-матрица).
- **plan-test-effort** — наивный, «первый проход» плана усилий по тестированию; критичность (critical/major/minor), лёгкий реестр рисков, веса S / M / L, открытые вопросы для углублённого прохода.
- **extend-coverage** — расширить покрытие, опираясь на существующие активы.
- **update-frd-and-tests** — протянуть обновление spec во внутривнутрипроектной FRD; вышестоящие системы (Confluence, Jira, ...) остаются доступны только для чтения.
- **manual-reproduction-guide** — repro, который может выполнить человек.
- **manage-backlog** — BACKLOG.md.
- **pr-report** — отчёт с учётом типа PR.

Рефакторинг (2)

- **refactor-fragmentation** — DRY на усмотрение пользователя.
- **introduce-pom-retries** — повторы внутри POM с разбивкой на два теста (first-try + with-retries).

Состояние (1)

- **question-state** — опросить окружение, прежде чем доверять результату.

Настройка (2)

- **setup-environment** — venv, инструменты разработки, набор скиллов Ocarina, скопированный в директорию скиллов Claude Code, пути к драйверам в CLAUDE.local.md, цикл pre-commit, smoke-проверка раннера.
- **profile-environment** — определяет рамки дозволенного на проекте (доступ к исходникам, зондирование вживую, чувствительность данных, конфиденциальность, безопасность, автономия, правки в репозитории) и генерирует дополнение CLAUDE.profile.md, которое лишь ужесточает настройки по умолчанию и никогда их не ослабляет.

Запуск (1)

- **propose-visual-review** — перед локальным запуском предлагает выбор: **--not-headless** (смотреть, как браузер отыгрывает сценарий) или headless (как в CI). Собирает команду; запускает пользователь.

Повторяющиеся цепочки

Набор не зелёный: `review-report` → `analyse-*` → `write-a-probe` → находка ложится в `IDENTIFIED_GAPS.md` / FRD / комментарий сценария → `probe` удаляется.

Сценарий чёрной шляпы выглядит многообещающе: `empiricism` → `extend-coverage` (часто intentional-fail).

Изменения в спес: `update-frd-and-tests` (сначала FRD, тесты следом). Gap-тесты переосмысляются, а не переворачиваются.

Нужен новый примитив Ocarina: сначала `understand-ocarina`, потом писать.

Собираетесь запустить прогон: `propose-visual-review` — с интерфейсом (`--not-headless`) или headless (как в CI)? Собирает команду; запускает пользователь.

Дисциплина

Показывайте, а не применяйте. Навыки выдают результат — решает пользователь.

Эмпирика, а не утверждения. Каждое утверждение о SUT — наблюдаемое, процитированное, датированное. Ритуальная фраза: *"Fair point, I'm assuming. Let me verify empirically."*

Gap-тесты переосмыляют, а не перекрашивают в зелёный. Инвертируйте утверждение, переименуйте, перенесите строку в strategy-doc, занесите решение в `IDENTIFIED_GAPS.md`. Одно движение — через `update-frd-and-tests`.

Эмиссии watcher'ов — только негативные сигналы. Watcher, выпускающий *"login succeeded"*, нарушает контракт.

Распределённо — когда дефицит общий. Если воркеры борются за ограниченный на стороне SUT ресурс (сессии, слоты, квоты), координируйте их через распределённые примитивы. Иначе worker-local in-memory cache вполне подойдёт — при условии, что ключи не могут столкнуться, а их генерация потокобезопасна.

Рамки можно только сужать. По умолчанию всё открыто, как на демо: чтение исходников, зондирование вживую, публичные учётные данные. `profile-environment` подстраивает их под конкретный проект, никогда не ослабляя правила безопасности.

Mtime, а не имя файла. UUID-суффиксы случайны; `pick-*` сортирует по mtime.

Чего эта схема не делает

- Не генерирует тесты автономно.
- Не замазывает галлюцинации в CI; сбой запускает `review-report` + `analyse-*`.
- Не переписывает спес; это делает только `update-frd-and-tests` — со строкой ревизии.
- Не запускает активные тесты безопасности. Никогда.

Открытые ресурсы

- <https://mojo-molotov.github.io/ocarina-holy-book/llms.txt>
- <https://mojo-molotov.github.io/ocarina-holy-book/llms-full.txt>
- <https://mojo-molotov.github.io/ocarina-holy-book/CLAUDE.md>

- <https://mojo-molotov.github.io/ocarina-holy-book/CLAUDE.slim.md>
- <https://mojo-molotov.github.io/ocarina-holy-book/ocarina-ru.pdf>
- <https://mojo-molotov.github.io/ocarina-holy-book/ocarina-en.pdf>
- <https://mojo-molotov.github.io/ocarina-holy-book/ocarina-fr.pdf>

[1] <https://github.com/mojo-molotov/ocarina-with-ai-example>

From Ocarina to Igor

Куда пойти, чтобы увидеть картину целиком.

Holy Book описывает саму Ocarina. Всё остальное (наборы-примеры, хаотичный SUT и обоснование каждого решения) описано в *From Ocarina to Igor*.

Чтобы пойти дальше

<https://mojo-molotov.github.io/from-ocarina-to-igor/>

Ресурс, открытый для LLM

Канонический машиночитаемый спутник — направьте свою модель прямо на него:

<https://mojo-molotov.github.io/from-ocarina-to-igor/llms.txt>